LAB- 04
Vibe coding session log: Meteor Dodge repair lab

Prompts used and the changes they produced while fixing the bug and adding the four tasks.

GitHub (all labs 1-4): https://github.com/akash-singh-1847u/SE-project

Name: Akash Singh
SRN: PES2UG24CS042
Section: A
Assignment: Lab 4 - VibeCoding (individual)
Submission repo: github.com/akash-singh-1847u/SE-project
Assigned repo: github.com/SETAPESU26/38-meteor-dodge
Game: Meteor Dodge (Pygame)
LLM used: Claude (Anthropic)

1. Starting point

I cloned the assigned repo and read the README. The game runs as provided: the ship launches on SPACE, meteors fall and tumble, and touching one ends the run. The README lists one deliberate bug and three optional features. Before changing anything I read the four source files (main.py, game/game_engine.py, game/ship.py, game/meteor.py) to find where each task would slot in.

My first prompt set the context for the whole session:

```
Here is a Pygame project called Meteor Dodge. It has one deliberate bug and four
tasks in the README. Read the four source files, tell me what the bug is, and
walk through each task one at a time. Keep the existing structure and style.
```

The reading confirmed the bug: in handle_events, pressing SPACE during active flight falls into the else branch and just re-sets self.started = True, so nothing happens. The ship has no way to fire. That is Task 1.

2. Task 1 - fix the input-state bug and fire lasers

Prompt:

```
During flight, SPACE does nothing because it only re-sets self.started. Make
SPACE fire an upward laser while flying, keep launch and restart as they are,
and have lasers destroy meteors they hit. Put the laser in its own module.
```

What changed:

A new game/laser.py holds a small Laser class that travels straight up and reports when it hits a meteor (distance to the meteor centre under the meteor radius). In the engine, the SPACE handler now branches three ways instead of two.

```
if event.key ==
pygame.K_SPACE:     if
self.game_over:
self.reset()     elif not
self.started:
self.started = True     else:
        self.fire_laser()   # was a no-op before; now fires in flight
```

Each update, lasers move, then every laser is checked against every meteor. A laser that connects is spent and the meteor is removed. Lasers that leave the top of the screen are dropped.

3. Task 2 - meteor splitting on impact

Prompt:

```
When a laser destroys a large meteor it should break into 2 or 3 smaller
fragments that spread outwards. Small meteors should just disappear. Reuse
the Meteor class for the fragments rather than writing a new one.
```

What changed:

The Meteor constructor now takes optional position, radius and velocity, so a fragment is just another Meteor built at the parent's location with a smaller radius and an outward velocity. A radius of 20 or more counts as large. A split() method returns the child fragments for a large meteor and an empty list for a small one, which is what makes small meteors dissolve.

```
def split(self, width):
if not self.is_large():
        return []                       # small meteor
dissolves     children = []     count = random.randint(2, 3)
child_radius = max(8, self.radius // 2)     base =
random.uniform(0, 360)     for i in range(count):
        angle = base + (360 / count) * i + random.uniform(-15, 15)
speed = random.uniform(2.5, 4.5)         cvx =
math.cos(math.radians(angle)) * speed         cvy =
abs(math.sin(math.radians(angle)) * speed) + 1.0
children.append(Meteor(width, x=self.x, y=self.y,
radius=child_radius, vx=cvx, vy=cvy))     return children
```

In the laser-meteor loop the engine now calls m.split(WIDTH) on a destroyed meteor and adds whatever comes back to the meteor list. Fragments are half-size, so they fall under the large threshold and dissolve the next time they are hit.

4. Task 3 - collectible shield power-up orbs

Prompt:

```
Add a drifting orb that spawns now and then and falls down the screen. If the
ship touches it, give the ship a temporary shield that absorbs one meteor hit
instead of ending the run. Show the shield on the ship and in the HUD.
```

What changed:

A new game/orb.py holds an Orb class that drifts down with a slight sideways motion and a pulsing glow. The engine spawns one every nine seconds. Collecting an orb sets a shield timer. Collision handling moved into a register_hit method so the shield logic lives in one place:

```
def register_hit(self):
if self.shield_timer > 0:
        self.shield_timer = 0          # absorb one collision
self.ship.shielded = False         self.streak = 0                #
a hit also breaks the streak         self.multiplier = 1     else:
        self.game_over = True
```

The ship draws a cyan ring while shielded, and the HUD shows the seconds remaining. The shield lasts eight seconds or until it takes a hit, whichever comes first.

5. Task 4 - consecutive survival multiplier

Prompt:

```
Score currently adds 1 every frame. Add a multiplier that goes up by one for
every 10 seconds survived without taking a hit, and reset it to 1x when a hit
happens. Add score and multiplier to the HUD.
```

What changed:

The engine tracks a streak counter in frames. The multiplier is one plus the number of full ten-second blocks in the streak, and score gains the multiplier each frame instead of a flat 1. Because register_hit zeroes the streak, an absorbed hit drops the player back to 1x. Time is now tracked separately from score so the survival clock stays honest.

```
self.streak += 1 self.multiplier = 1 + self.streak // MULTIPLIER_STEP   #
MULTIPLIER_STEP = 10 * FPS self.score += self.multiplier
```

6. Checking it works

I drove the engine headlessly to confirm each task without needing the window open: a laser spawns on SPACE in flight; a large meteor splits into two or three fragments while a small one leaves nothing; an orb grants a shield, the shield absorbs one hit with no game over, and the hit after that ends the run; and the multiplier climbs to x2 after ten seconds and x3 after twenty, then drops back to x1 once a hit lands. A longer run with lasers firing and meteors, orbs and fragments all on screen at once ran without errors. After that I ran the game normally to record the after video.

Files added: game/laser.py, game/orb.py. Files changed: game/game_engine.py, game/meteor.py, game/ship.py. No change to main.py or requirements.txt.